6. `evaluation.py` expliqué

Nutri-Arbitre — fiche de révision · la mesure du système entier · **brique 9** (une comparaison mesurée)

> **En une phrase :** ce fichier pose **16 questions dont on connaît la bonne réponse**, 3 fois chacune, à deux systèmes : le **système complet** (Qwen + outils + garde-fou) et le **modèle seul** (Qwen sans outils ni garde-fou). Il juge chaque réponse (juste, fausse ou bloquée) et calcule le taux de réponses justes, avec sa moyenne et sa dispersion.
> **Brique 9** : au moins 10 tâches, au moins 3 lancements chacune, moyenne et dispersion, et un échec expliqué à partir de sa trace.
> **Lancer :** `python -m nutri.evaluation` (environ 35 minutes, 96 réponses de Qwen) ou `--rapide` (2 questions, 1 lancement, 1 minute).

Les réglages (lignes 24 à 26)

```
24 N_LANCEMENTS = 3
25 TOLERANCE = 0.02
26 DOSSIER = Path("resultats")
```

- **3 lancements** : Qwen ne répond pas deux fois pareil. Une seule fois ne dirait rien de sa régularité.
- **2 %** : un nombre de la réponse est juste s'il est à 2 % près de la bonne réponse (13 pour 13,17 : 1,3 % d'écart, juste).
- Les résultats sont écrits dans `resultats/evaluation.md` (le tableau) et `resultats/evaluation.json` (toutes les traces).

`construire_taches` : les 16 questions (lignes 29 à 82)

Chaque question est un petit dictionnaire : la question, le **type** de réponse attendue, et la bonne réponse (`attendu`).

> **Les bonnes réponses sont calculées directement avec pandas**, par exemple `bio["sucres"].mean()`, **sans passer par `tools.py`**. Si un outil avait une erreur de calcul, l'évaluation la verrait : c'est une vérification **indépendante**.

Type	Questions	Bonne réponse
`nombre`	combien de céréales, moyenne de sucres, de sel, de calories…	un nombre (ex. 953, 13,17)
`texte`	la marque la moins sucrée	« jardin bio etic »
`prudence`	les bio ont-elles plus de lipides ?	« on ne peut pas conclure » : l'écart 0,21 g est plus petit que sa marge 0,84 g

`refus`	la vitamine C	aucun chiffre : la donnée n'existe pas dans le fichier

Les 4 questions pièges (lignes 67 à 81)

Les 12 premières questions demandent chacune un seul outil : elles sont faciles. On a ajouté 4 questions qui visent les failles connues du système, parce que le but est aussi de **montrer ses limites** :

- **une portion de 30 g** : aucun outil ne fait ce calcul. Qwen calcule lui-même, le garde-fou bloque ;
- **l'écart de sucres entre Nutri-Score A et E** : l'outil donne les deux moyennes, mais la soustraction est faite par Qwen ;
- **la vitamine C** : elle n'est pas dans les données. La bonne réponse est de le dire, sans inventer ;
- **« les céréales sont-elles riches en sucres ? »** : la question ne parle pas de bio, mais Qwen est tenté de comparer bio et non bio (la règle n° 2 du garde-fou).

La marque la moins sucrée (lignes 36 et 37)

```
36      par_marque = df.groupby("marque")["sucres"].agg(["count", "mean"])
37      marque_moins_sucree = par_marque[par_marque["count"] >= 5]["mean"].idxmin()
```

Pour chaque marque : le nombre de produits et la moyenne de sucres. On garde les marques d'au moins 5 produits (le masque), puis `idxmin()` donne le **nom** de la ligne où la moyenne est la plus petite.

`nombre_juste` (lignes 85 à 94)

```
89      ecart_permis = max(TOLERANCE * abs(attendu), 0.01)
90      for ecrit in garde_fou.extraire_nombres(reponse):
91          valeur = abs(garde_fou.en_float(ecrit))
92          if abs(valeur - abs(attendu)) <= ecart_permis:
93              return True
94      return False
```

- On **réutilise** le garde-fou pour trouver les nombres de la réponse (la même expression régulière).
- L'écart permis est 2 % de la bonne réponse, mais **au moins 0,01**. Pourquoi : le sel bio vaut 0,1265. L'outil l'arrondit à « 0,13 », qui est à 0,0035 ; or 2 % de 0,1265, c'est seulement 0,0025. Sans ce minimum, une réponse juste serait comptée fausse.
- `abs` enlève le signe : un écart de −5,42 peut s'écrire « 5,42 g de moins ».
- Il suffit qu'**un** nombre de la réponse soit juste.

`juger` : le verdict (lignes 97 à 113)

```
99      if trace["bloquee"]:
100         return "bloquee"
102     if tache["type"] == "nombre":
103         ok = nombre_juste(reponse, tache["attendu"])
104     elif tache["type"] == "texte":
105         ok = tache["attendu"].lower() in reponse.lower()
106     elif tache["type"] == "refus":
107         ok = garde_fou.verifier(reponse, [], trace["question"])[0]
108     else:   # prudence
110         for mot in tache["attendu"]:
111             if mot in reponse.lower():
112                 ok = True
```

- **On regarde d'abord si la réponse est bloquée**, et c'est important : le message de refus contient les nombres refusés (« … (122,84) »). Sans cette ligne, un refus pourrait être compté « juste » par accident. Une réponse bloquée n'est ni juste ni fausse : le système n'a **rien affirmé**.
- **texte** : le nom de la marque est-il dans la réponse ? `.lower()` met tout en minuscules, pour que « Jardin Bio Etic » compte aussi.
- **refus** : on passe la réponse dans le garde-fou **sans aucun résultat d'outil**. Si elle contient un nombre qui ne vient pas de la question, elle a inventé : fausse.
- **prudence** : la réponse contient-elle un des mots « hasard », « pas conclure »… ?

`resumer` : moyenne et dispersion (lignes 116 à 129)

```
119     taux = []
120     for lancement in verdicts:
121         taux.append(lancement.count("juste") / len(lancement))
126             "taux_juste_moyen": round(statistics.mean(taux), 3),
127             "ecart_type": round(statistics.stdev(taux), 3) if len(taux) >= 2 else
None,
```

- Pour chaque lancement : le **taux de réponses justes** = nombre de « juste » / nombre de questions. Ex. : 14 justes sur 16 = 0,875, soit 87,5 %.
- **La moyenne** des 3 taux, et leur **écart type** : il mesure de combien les taux changent d'un lancement à l'autre. 0 % veut dire : les 3 lancements ont donné le même score.
- `stdev` a besoin d'au moins 2 valeurs (on ne mesure pas la dispersion d'un seul nombre) : d'où le `None` en mode rapide.
- Lignes 122 à 124 : on met bout à bout les verdicts des 3 lancements pour compter les justes, les fausses et les bloquées au total.

`evaluer` : la grande boucle (lignes 132 à 163)

```
139     for lancement in range(n_lancements):
142         for i, tache in enumerate(taches):
143             for nom, avec_outils in systemes.items():
144                 trace = repondre(tache["question"], df, avec_outils=avec_outils)
145                 verdict = juger(tache, trace)
```

- Trois boucles l'une dans l'autre : 3 lancements × 16 questions × 2 systèmes = **96 réponses**.
- `repondre` est la fonction d'`agent.py` : l'évaluation teste **le vrai système**, exactement comme dans le terminal. `avec_outils=False` donne le modèle seul.
- `enumerate` donne le numéro de la question (`i`) en plus de la question.
- Chaque trace est gardée dans `details` : c'est là qu'on va chercher un échec pour l'expliquer.
- `construire_taches(df)[:n_taches]` (ligne 134) : on garde les `n_taches` premières questions. Avec `None` (par défaut), `[:None]` les garde toutes.
- Lignes 151 à 160 : on range les verdicts par question (ex. `["juste", "juste", "bloquee"]`) pour le tableau.

Les résultats (3 novembre 2026, 96 réponses)

Système	Réponses justes (moyenne ± écart type)	Justes	Fausses	Bloquées	Durée
Complet	**73 % ± 4 %**	35	5	8	19 s
Modèle seul	**12 % ± 6 %**	6	42	0	13 s

- **Les outils changent tout** : sans eux, Qwen ne connaît pas nos données. Il dit « j'ai besoin d'un outil », ou il **invente** (17 réponses sur 48), souvent avec une fausse marge « ± ».
- **Le garde-fou a bloqué 8 réponses**, toutes à raison : 5 calculs faits par Qwen (portion, écart A / E), un « 4,42 g de moins » au lieu de 5,42, et 2 réponses de mémoire.
- Une première évaluation sans les pièges avait donné **100 %** au système complet : c'est pour ça qu'on les a ajoutés.

L'échec expliqué (lancement 1, question 11, système complet) : « Quelle marque (au moins 5 produits) a la teneur moyenne en sucres la plus basse ? » Bonne réponse : jardin bio etic, 6,5 g.

1. Qwen appelle `classement_marques(sucres, ordre="decroissant")` : l'outil renvoie les marques les **plus** sucrées en premier ;
2. il lit la première comme la moins sucrée : « Crownfield, 18,61 g » ;
3. le garde-fou laisse passer : 18,61 vient d'un outil. **Il vérifie l'origine des chiffres, pas le sens.**

Fausse 3 fois sur 3, alors que 30 minutes plus tôt, la même question était juste 3 fois sur 3 (avec `ordre="croissant"`) : Qwen change d'un passage à l'autre.

> **Les faux positifs, gardés volontairement.** Pour les lipides, le modèle seul est compté juste 3 fois sur 3 : il invente des moyennes (« 3,2 g ± 0,4 g » au lieu de 10,69 g) mais écrit « pas significatif ». Le jugement par mots-clés ne vérifie pas les chiffres. **Ma décision** : ne pas corriger le jugement après avoir vu les résultats, et montrer cette erreur de la mesure. Malgré les outils, le garde-fou et le prompt, le système se trompe, et la mesure aussi.

Questions d'examinateur

> 1. Que compare la brique 9, et pourquoi ces deux systèmes ?
> 2. Comment connaissez-vous les bonnes réponses ? Pourquoi ne pas utiliser `tools.py` pour les calculer ?
> 3. Pourquoi poser chaque question 3 fois ?
> 4. Que veut dire « 2 % près », et pourquoi un minimum de 0,01 ?
> 5. Pourquoi tester `trace["bloquee"]` en premier dans `juger` ?
> 6. Une réponse bloquée est-elle un échec ?
> 7. Que mesure l'écart type ici ?
> 8. Pourquoi avoir ajouté des questions pièges ?
> 9. Racontez un échec à partir de sa trace.
> 10. Quelles sont les limites de cette mesure ?

Corrigé

1. Le **système complet** (Qwen + outils + garde-fou) et le **modèle seul** (Qwen sans outils). On mesure ce qu'apportent les outils et le garde-fou, en justesse et en coût (temps, tokens).
2. Calculées directement avec pandas dans `construire_taches`. C'est une vérification indépendante : si `tools.py` avait une erreur, on la verrait, alors qu'avec les outils on comparerait le code à lui-même.
3. Qwen ne répond pas deux fois pareil. Avec 3 lancements, on peut calculer une moyenne et une dispersion (la consigne en demande au moins 3).
4. Un nombre est juste s'il s'écarte de moins de 2 % de la bonne réponse (13 pour 13,17). Le minimum de 0,01 vient des arrondis à 2 décimales des outils : 0,13 pour un sel de 0,1265 doit être juste.
5. Le message de refus contient les nombres refusés. Sans ce test, un refus qui contient par hasard le bon nombre serait compté juste.
6. Pas une erreur : le système n'a rien affirmé de faux, il a refusé. Mais ce n'est pas une réussite non plus : l'utilisateur n'a pas de réponse. C'est pour ça qu'on la compte à part.
7. De combien le taux de réponses justes change d'un lancement à l'autre : la régularité du système.
8. Les 12 premières questions demandent un seul outil : le système les réussit toutes. Les pièges montrent ses failles connues (calcul fait par le modèle, donnée absente, filtre non demandé). Le but du devoir est de mesurer honnêtement, pas d'obtenir 100 %.
9. Question 11, système complet : Qwen demande le classement des marques en ordre **décroissant**, puis lit la première (la plus sucrée, Crownfield, 18,61 g) comme la moins sucrée. Le garde-fou laisse passer, parce que 18,61 vient d'un outil : il ne vérifie pas le sens de la phrase.
10. 16 questions et 3 lancements, c'est peu (la question 11 a changé d'un passage à l'autre) ; le jugement par mots-clés ne lit pas le sens (les faux positifs des lipides, gardés volontairement) ; le modèle seul garde un prompt qui parle d'outils qu'il n'a pas ; les questions sont écrites par nous.